

Flow of Control

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Branching structure
- Loop structure
- Algorithm efficiency
- Exercises

Branching structure

Boolean

- Applied to "things" that have two values
 - ON/OFF, open/close, true/false
- It contains **true** or **false**
- In C++, denoted as `bool`, in an indirect way
 - `false`: value 0
 - `true`: value not 0
- For example:
 - 0 (`false`), 1 (`true`), -1 (`true`), 2 (`true`)
 - 1 < 2 (`true`), 3 > 1 (`true`)

Relational Operators

- Check the relationship between two operands

- Return **true** or **false**

- In C++,

- **==**, **!=**, **>**, **<**, **>=**, **<=**

- For example:

s1 = (1 == 2) ; s3 = (1 > 2) ; s5 = (1 < 2) ;

s2 = (1 != 2) ; s4 = (1 >= 2) ; s6 = (1 <= 2) ;

Logical Operators

- Check whether an expression is **true** or **false**

- In C++,

and **& &**,

or **| |**,

not **!**

- For example

s1 = (1 > 2) **& &** (3 > 4) ;

s2 = (1 > 2) **| |** (3 > 4) ;

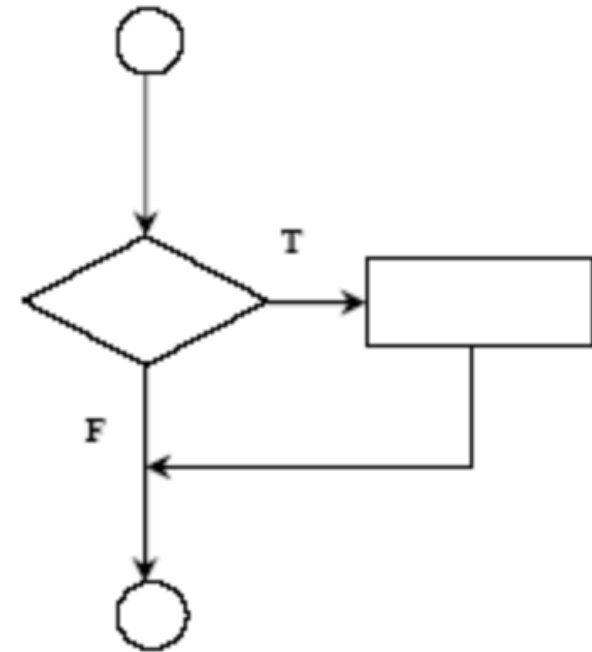
s3 = **!** (1 > 2) ;

&&	0	1
0	0	0
1	0	1
 	0	1
0	0	1
1	1	1

if-statement (one-way)

```
if (<Condition>)  
    <Statement>;
```

```
if (<Condition>) {  
    <Statement1>;  
    <Statement2>;  
}
```



if structure – single selection

if-statement (one-way)

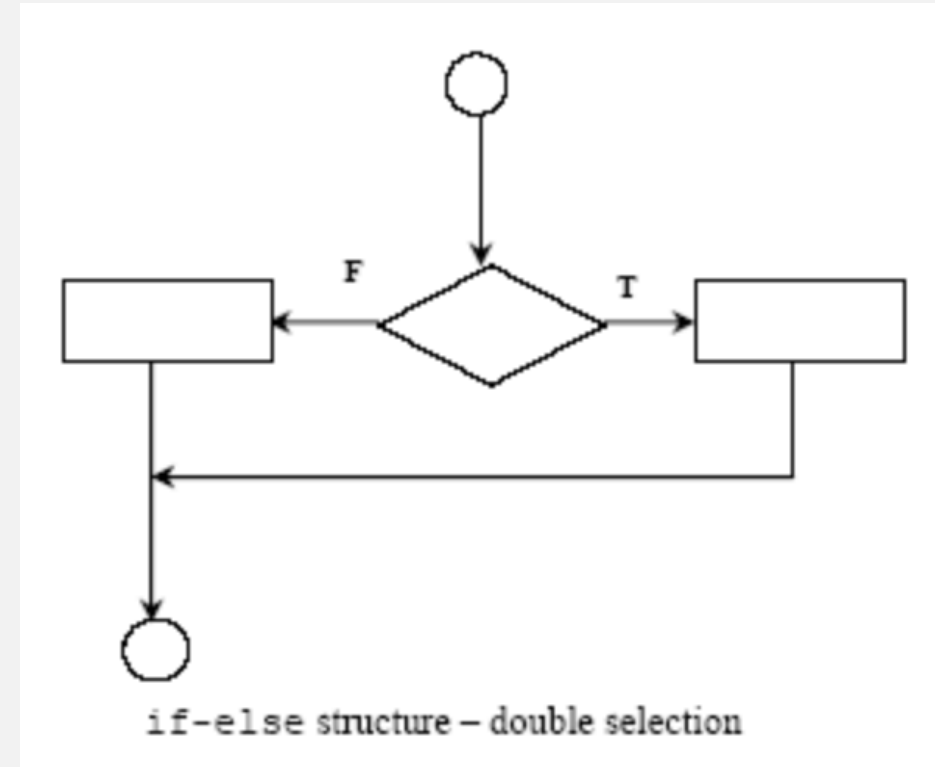
```
int main() {  
    int a = 0;  
    if (a == 0)  
        cout << "a = 0\n";  
    if (a == 0) {  
        cout << "a = 0\n";  
        a++;  
    } // result of a?  
    return 0;  
}
```

if-statement (complete)

```

if (<Condition>)
    <Statement>;

else {
    <Statement1>;
    <Statement2>;
}
    
```



if-statement (complete)

```
int main() {  
    int a = 0;  
    if (a == 0)  
        cout << "a is equal to 0\n";  
    else { // case a != 0  
        cout << "a is not equal to 0\n";  
        a++;  
    }  
    return 0;  
}
```

Notes

- **if**-statement can be **nested** and **else**-statement will be associated with the most recent **if**

```
if (a != 0)
```

```
    if (b > 0)
```

```
        cout << "a != 0 and b > 0\n";
```

```
else
```

```
    cout << "a != 0 and b <= 0\n";
```

Notes

- **if**-statement can be **nested** and **else**-statement will be associated with the most recent **if**

```
if (a != 0) {  
    if (b > 0)  
        cout << "a != 0 and b > 0\n";  
    else  
        cout << "a != 0 and b <= 0\n";  
}
```

Notes

- Should use **else** to eliminate cases

```
if (delta < 0)
    cout << "No solution\n";
```

```
if (delta == 0)
    cout << "There is one solution\n";
```

```
if (delta > 0)
    cout << "There are two solutions\n";
```

Should not
use this way

Notes

- Should use **else** to eliminate cases

```
if (delta < 0)
    cout << "No solution\n";
else // delta >= 0
    if (delta == 0)
        cout << "There is one solution\n";
    else
        cout << "There are two solutions\n";
```

Should use
this way

Notes

- Distinguish `==` and `=` when using `if`

```
if (x = 5)    // Can the program be run?  
    cout << "The value is five." << endl;
```

```
if (x == 5)  
    cout << "The value is five." << endl;
```

Ternary Operator (?:)

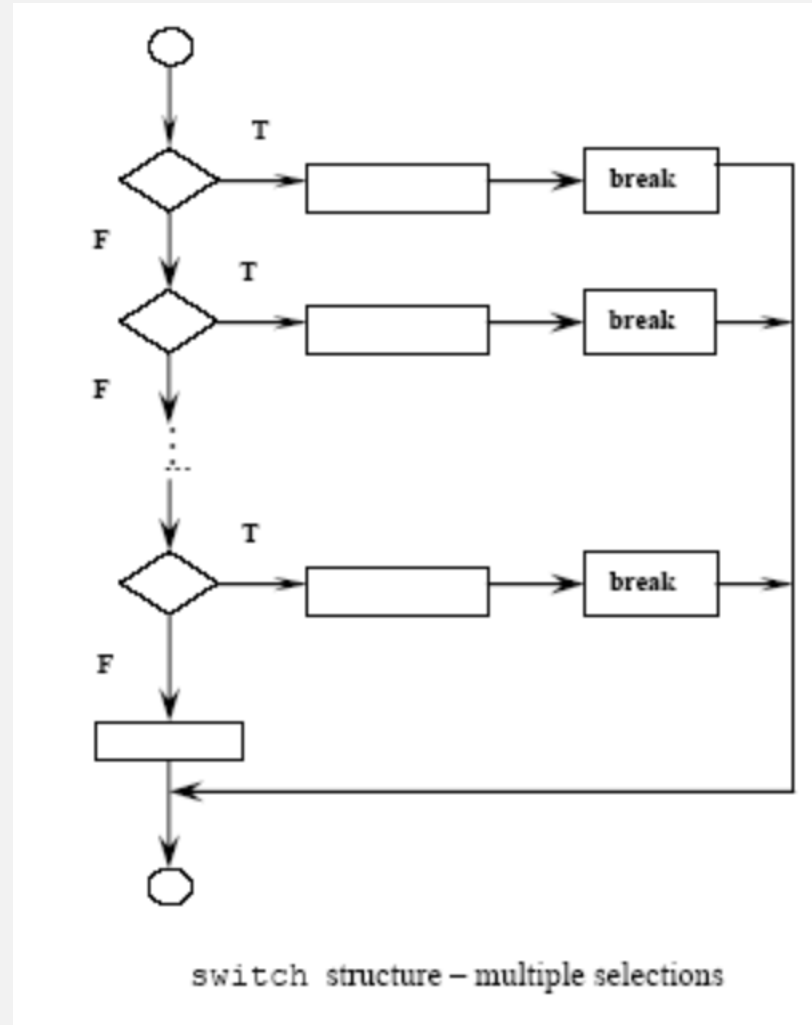
- Operator **?:** takes 3 operands

`<Condition> ? <Expression1> : <Expression2>;`

```
int Score = 51;
```

```
int Result = (Score <= 50) ? 0 : 1; // assume  
that 0 and 1 are fail and pass, respectively.
```

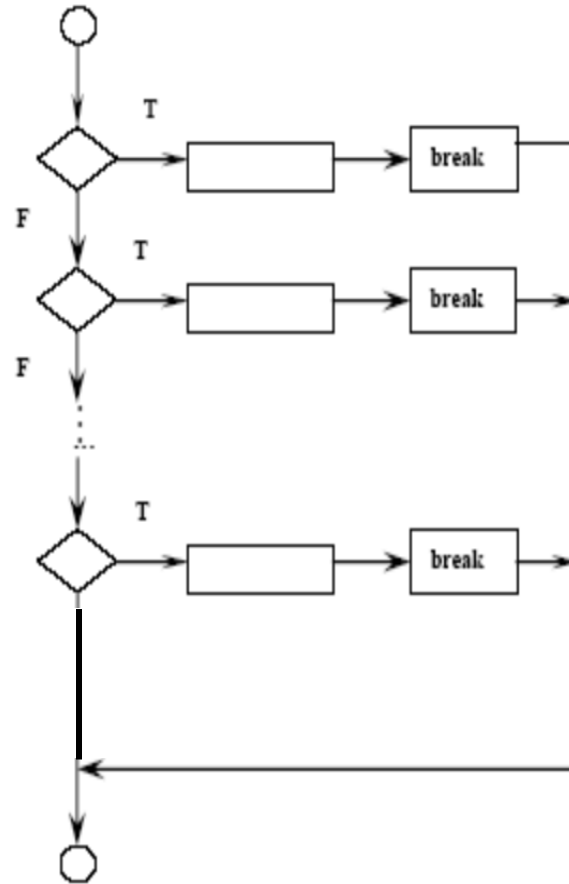
switch-statement



switch-statement (not complete)

```
switch (<Variable/Expression>) {
    case <Value1>:<Statement1>; break;
    case <Value2>:<Statement2>; break;
    ...
}
```

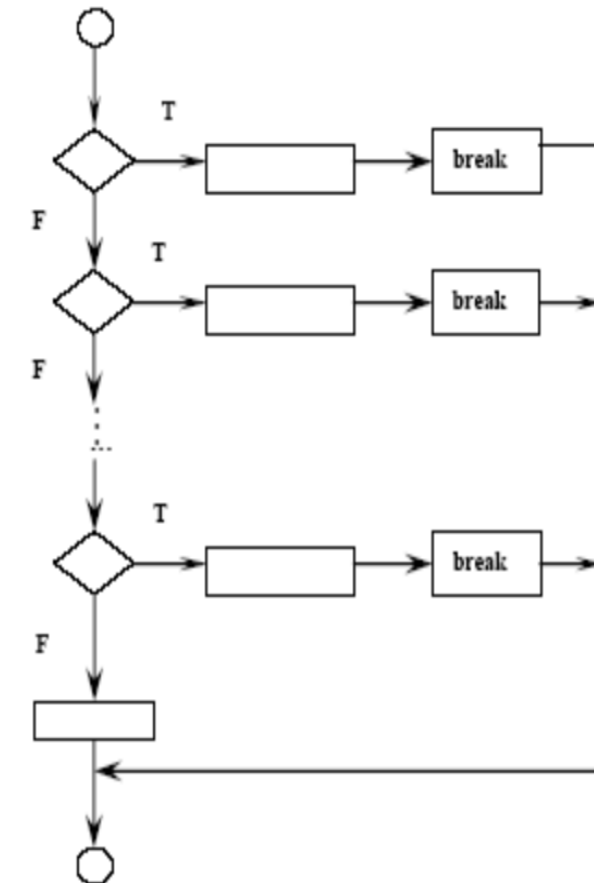
```
switch (a) {
    case 1: cout << "One\n"; break;
    case 2: cout << "Two\n"; break;
    case 3: cout << "Three\n"; break;
}
```



switch structure – multiple selections

switch-statement (complete)

```
switch (<Variable/Expression>) {
    case <Value1>:<Statement1>; break;
    case <Value2>:<Statement2>; break;
    ...
    default:
        <StatementN>;
}
```



switch structure – multiple selections

switch-statement (complete)

```
int main() {  
    int a;  
    cout << "Input a: "; cin >> a;  
    switch (a) {  
        case 1: cout << "One\n"; break;  
        case 2: cout << "Two\n"; break;  
        case 3: cout << "Three\n"; break;  
        default: cout << "Cannot pronounce\n";  
    }  
}
```

Notes

- Case in **switch**-statement is a single statement and can be nested.

```
switch (a) {  
    case 1: cout << "One\n"; break;  
    case 2: switch (b) {  
        case 1: cout << "A\n"; break;  
        case 2: cout << "B\n"; break;  
    } break;  
    case 3: cout << "Three\n"; break;  
    default: cout << "Cannot pronounce\n";  
}
```

Notes

- Each value (of cases) in **switch**-statement must be **different**

```
switch (a) {  
    case 1: cout << "One\n"; break;  
    case 1: cout << "ONE\n"; break;  
    case 2: cout << "Two\n"; break;  
    case 3: cout << "Three\n"; break;  
    case 1: cout << "1\n"; break;  
    case 1: cout << "one\n"; break;  
    default: cout << "Cannot pronounce\n";  
}
```

Notes

- Each value (of cases) in **switch**-statement must be **different**

```
switch (a) {  
    case 1: cout << "One\n"; break;  
case 1: cout << "ONE\n"; break;  
    case 2: cout << "Two\n"; break;  
    case 3: cout << "Three\n"; break;  
case 1: cout << "1\n"; break;  
case 1: cout << "one\n"; break;  
    default: cout << "Cannot pronounce\n";  
}
```

Notes

- **switch** will move on to cases until meeting **break** or final statement of **switch**.

```
switch (a) {  
    case 1: cout << "One\n"; break;  
    case 2: cout << "Two\n"; break;  
    case 3: cout << "Three\n"; break;  
}
```

Notes

- We can utilize this feature when removing `break`;

```
switch (a) {  
    case 1:  
    case 3: cout << "Odd number\n"; break;  
    case 2:  
    case 4: cout << "Even number\n"; break;  
}
```

Practice: Writing a program that receives a month. The program returns a number of days in the month.

Notes

- Using **switch** when values are **discrete values**
 - `double` numbers must not be used in **switch**
- **switch** is often used with **enum** datatype

Loop structure

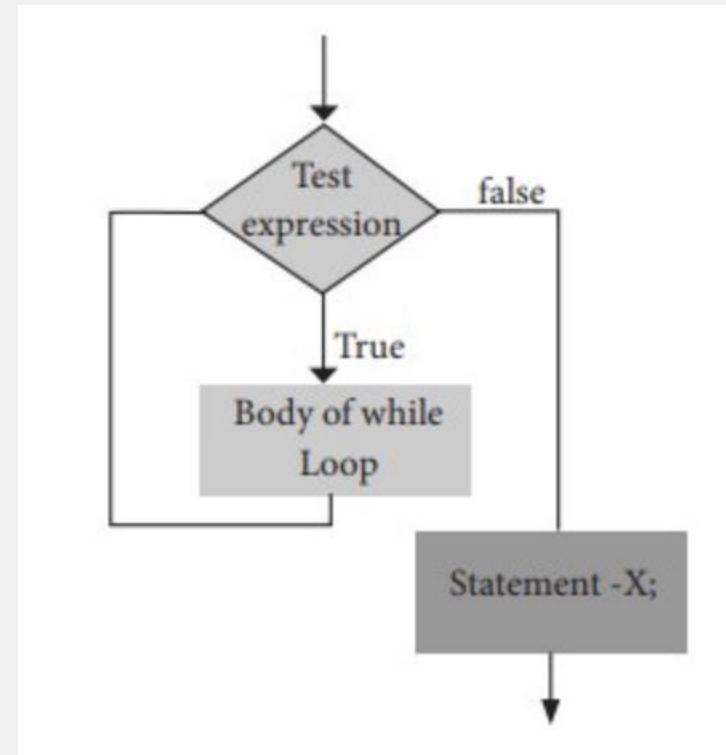
Motivation

- Write a program to print out numbers from 1 to 10
 - One possible idea is to use 10 output statements (**cout**)
- Write a program to print out numbers from 1 to 1000 (or a positive arbitrary number)
 - Can we use 1000 output statements?
- Using **loop structure** for action(s) when it meets condition(s)
- In C++,
 - **while**,
 - **do ... while**,
 - **for**

while statement

```
while (<LoopCondition>)  
    <Statement>;
```

```
while (<LoopCondition>) {  
    <Statement1>;  
    <Statement2>;  
    ...  
}
```



while statement

```
int main() {  
    // Print out numbers from 1 to 10  
    int i = 1;                // initialization  
    while (i <= 10) {  
        cout << i << endl;  
        i++;                  // increment  
    }  
    return 0;  
}
```

Notes

- **while**-statement is a single statement and can be nested.

```
if (n < 10 && m < 20)
{
    while (n >= 1) {
        while (m >= 1) {
            cout << m << endl; m--;
            n--;
        }
    }
}
```

Notes

- Statements in **while** can **never be run** because **loop condition is not statified** at the first time

```
int main() {  
    int n = 1;  
    while (n > 10) {  
        cout << n << endl; n--;  
    }  
  
    return 0;  
}
```

Notes

- Do not add **;** after **while**

```
int n = 0;
while (n < 10) ; {
    cout << n << endl; n++;
}
```

```
while (n < 10) {
    // do something
} ;
```

Notes

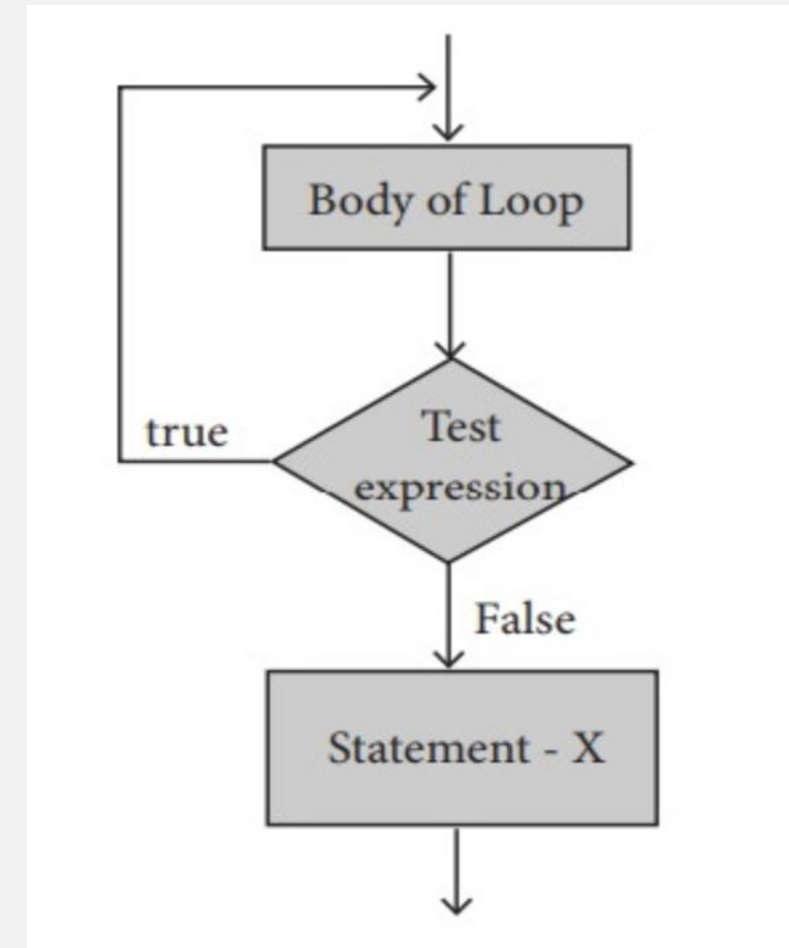
- Statements in **while** can be looped forever

```
int n = 1;
while (n < 10) {
    cout << n;
    n--;
}
// n >= 10?

n = 1;
while (n < 10)
    cout << n;
// n >= 10?
```

do ... while statement

```
do
    <Statement>;
while (<LoopCondition>) ;
do {
    <Statement1>;
    <Statement2>;
    ...
}
while (<LoopCondition>) ;
```



do ... while statement

```
// Print out numbers from 1 to 10
int i = 1;
do{
    cout << i << endl;
    i++;
}
while (i <= 10);
```

Notes

- **do... while** statement is a single statement and can be nested

```
int a = 1, b;
```

```
do {
```

```
    b = 1;
```

```
    do {
```

```
        cout << a + b << endl;
```

```
        b = b + 2;
```

```
    }
```

```
    while (b < 20); a++; // a++ for which do ... while?
```

```
} while (a < 20);
```

Notes

- Statements in **do... while** run at least one time because the loop condition is checked in last statement

```
int main() {  
    int n;  
    do{          // check the valid input  
        cout << "Input n: ";  
        cin >> n;  
    } while (n < 1 || n > 100);  
    return 0;  
}
```

Notes

- Statements in **do... while** can be looped forever

```
int n = 1;
do{
    cout << n; n--;
} while (n < 10);           // n >= 10?
```

```
n = 1;
do
    cout << n;
while (n < 10);           // n >= 10?
```

while & do... while

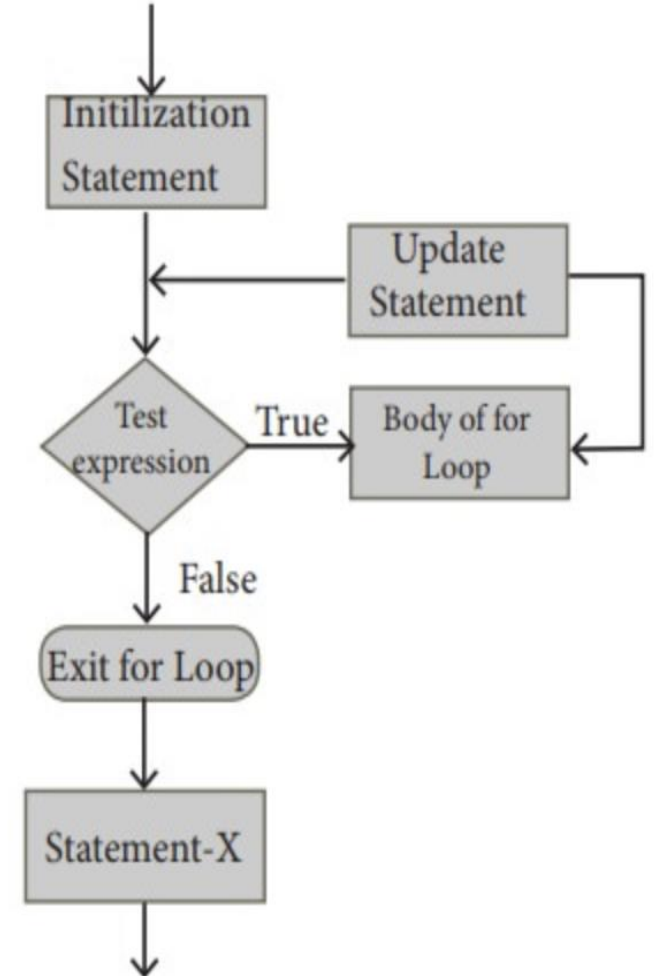
- Statements in **while** may not be run once
- Statements in **do... while** run at least one time

```
int n = 100;
while (n < 10) {
    // never be run
}
do
{
    // run at least one time
    cout << "input n: "; cin >> n;
} while (n > 10);
```

for statement

```
for (<Init> ; <LoopCondition> ; <Steps>)
    <Statement>;
```

```
for (<Init> ; <LoopCondition> ; <Steps>) {
    <Statement1>;
    <Statement2>;
    ...
}
```



for and while statements

```
int i = 0;
while (i < 10) {
    cout << i << endl; i++;
}
// equivalent to (not same)
for (int i = 0; i < 10; i++)
    cout << i << endl;
```

Notes

- **for**-statement is a single statment and can be nested

```
if (n < 10 && m < 20) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            cout << i + j << endl;  
        }  
    }  
}
```

Notes

- **for**-statement is a single statment and can be nested

```
if (n < 10 && m < 20)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            cout << i + j << endl;
```

Notes

- **for**-statement may not have <Init> and <Steps>

```
int i = 0;
```

```
while (i < 10) {  
    cout << i << endl; i++;  
}
```

```
int i = 0;
```

```
for (; i < 10;) {  
    cout << i << endl;  
    i++;
```

```
}
```

Notes

- Even not have <LoopCondition>

```
int i = 0;
```

```
while (i < 10) {
```

```
    cout << i << endl; i++;
```

```
}
```

```
for (int i = 0; ; i++) {
```

```
    if (i < 10)
```

```
        cout << i << endl;
```

```
}
```

Notes

- Parts `<Init>`, `<LoopCondition>`, `<Steps>` are separated by **`;`**
- If each part contains many components, such components will be separated by **comma** **`,`**

```
for (int i = 1, j = 2; i + j < 10; i++, j += 2)
    cout << i + j << endl;
// How about comma in <LoopCondition>?
```

Notes

- Do not add **;** just after **for**
→ Equivalent to null statement (NOP)

```
for (i = 0; i < 10; i++) ; {  
    cout << i << endl;  
}
```

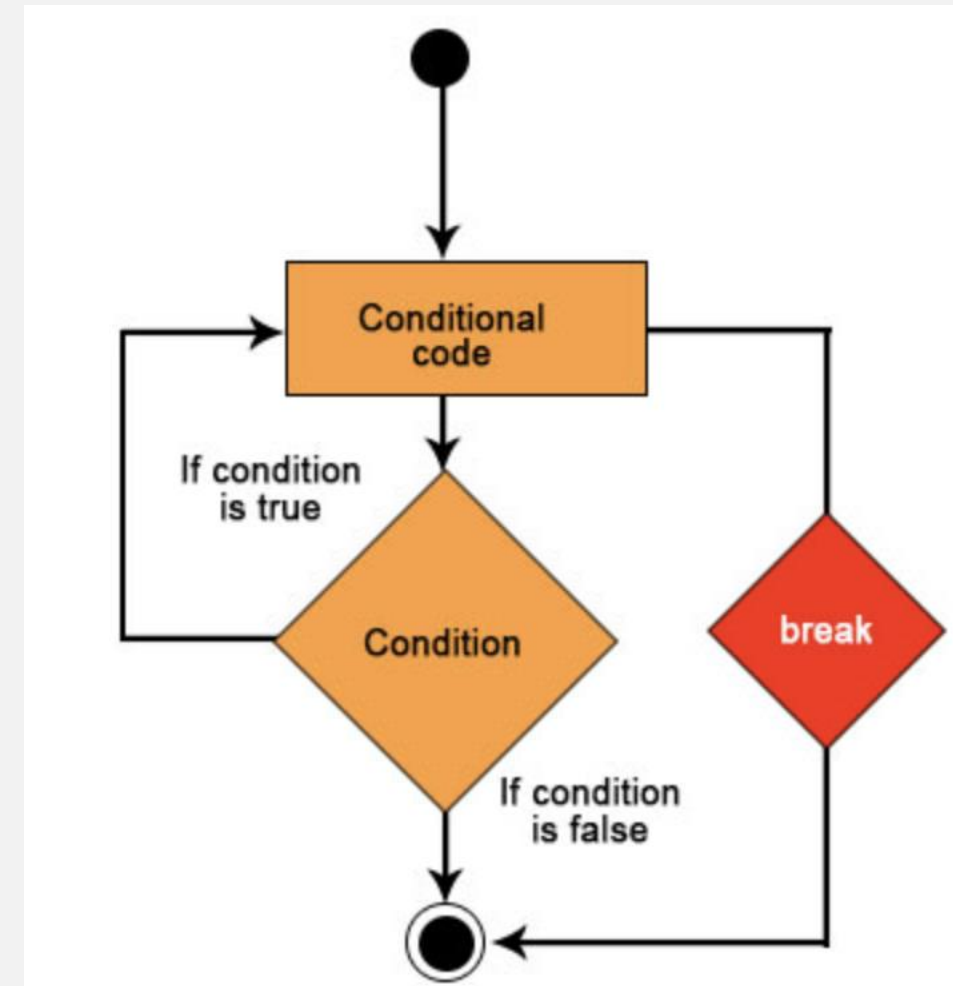
```
for (i = 0; i < 10; i++) {  
    ; {  
        cout << i << endl;  
    }
```

Notes when using loop structure

- **break**-statement terminates the nearest loop.

- Can apply to **while** and **do... while** // results?

```
for (i = 0; i < 10; i++) {
    if (i % 2 == 0)
        break;
    cout << i << endl;
}
```

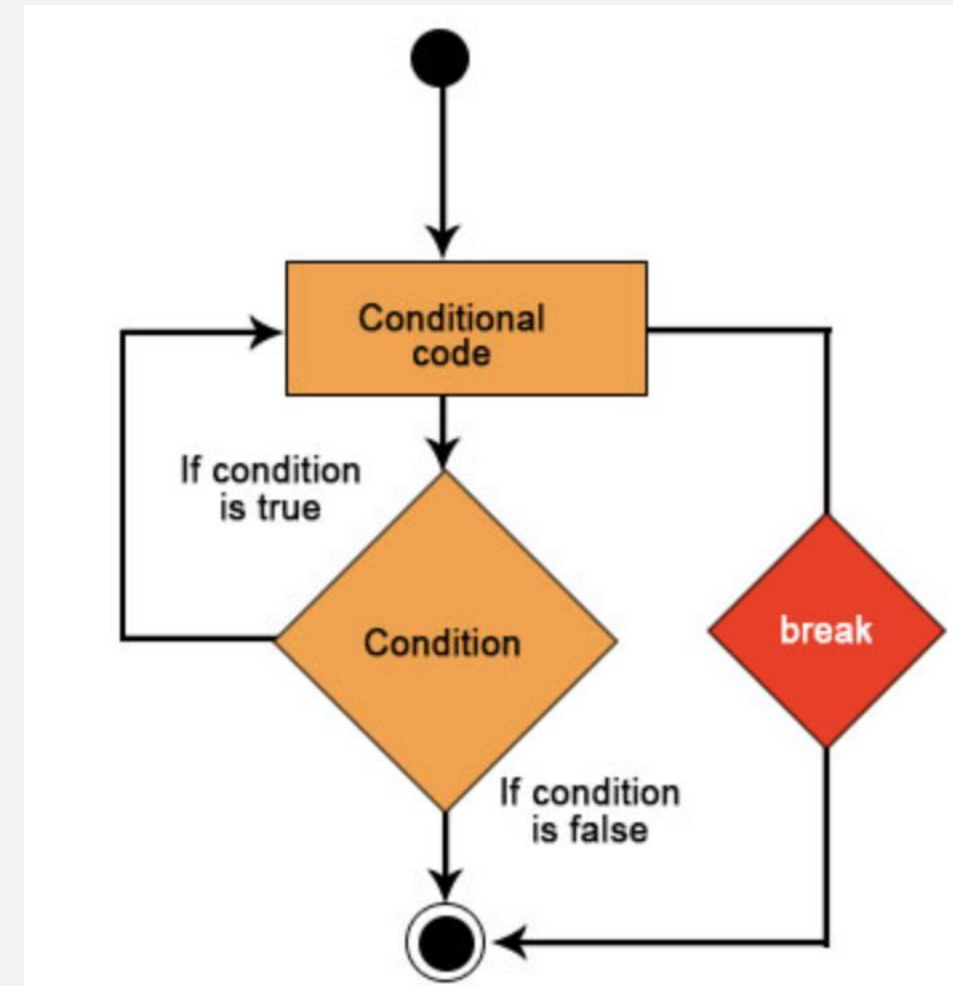


Notes when using loop structure

- **break**-statement terminates the nearest loop.

- Can apply to **while** and **do... while** // results?

```
for (i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        break;
    }
    cout << i << endl;
}
```



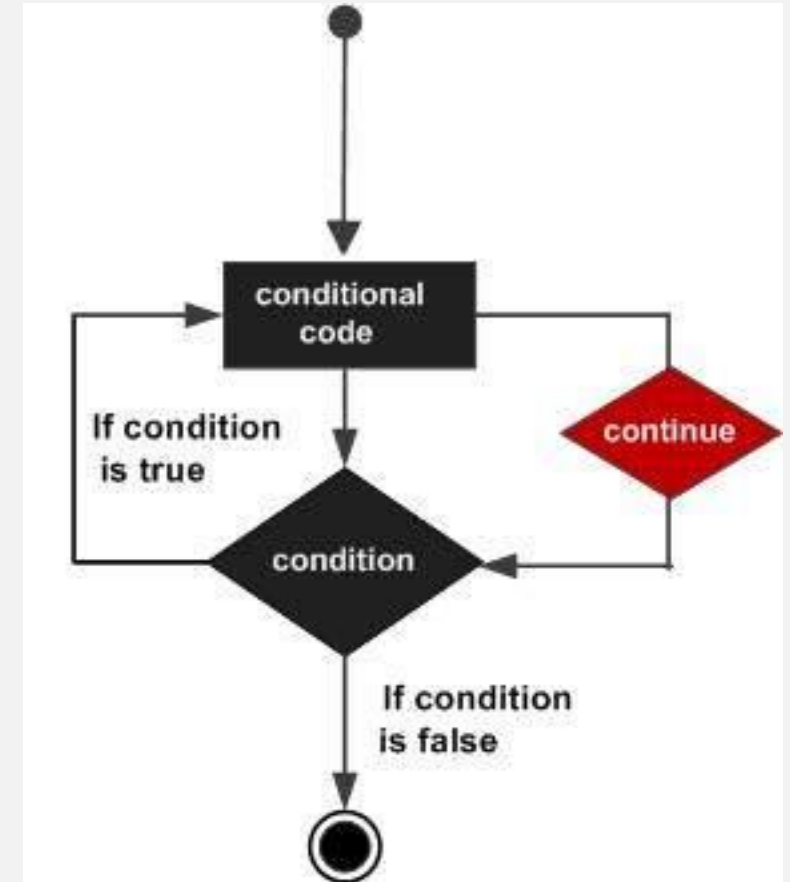
Notes when using loop structure

- **continue**-statement skips remaining statements and moves to next iteration

- Can apply to **while** and **do... while**

// results?

```
for (i = 0; i < 10; i++) {
    if (i % 2 == 0)
        continue;
    cout << i << endl;
}
```

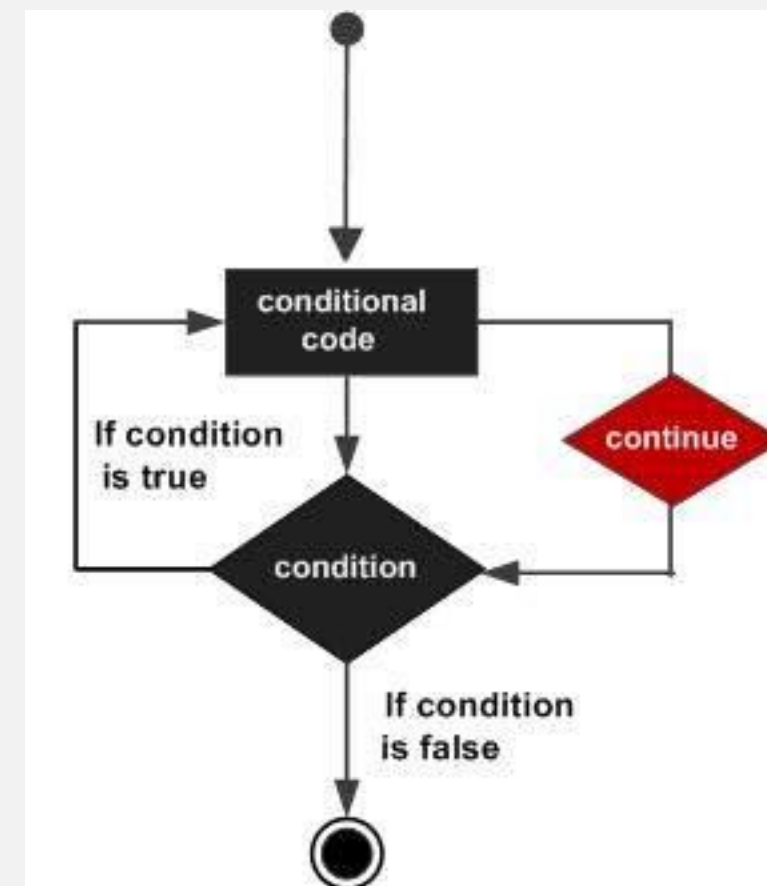


Notes when using loop structure

- **continue**-statement skips remaining statements and moves to next iteration

- Can apply to **while** and **do... while**
// results?

```
for (i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    cout << i << endl;
}
```



Notes when using loop structure

- If do not have **loop condition**, loop structure **can be repeated forever**
- Can **repeat** actions
- If **knowing** (loop) steps, use **for**
- If **not knowing** (loop) steps, use **while**
- If **not knowing** (loop) steps but knowing **run at least once**, use **do ... while**

Algorithm efficiency

Example 1

Given a positive integer n , check whether n is a prime number?

```
int i, n; cin >> n;
bool flag = true;
for (i = 2; i < n; i++)
    if (n % i == 0)
        flag = false;
if (flag == false)
    ...
```

```
int i, n; cin >> n;
bool isPrime = true;
for (i = 2; i < n; i++)
    if (n % i == 0) {
        isPrime = false;
        break;
    }
if (isPrime == false)
```

Example 1(cnt.)

```
... // Declaration
for (i = 2; i <= sqrt(n) ; i++)
    if (n % i == 0) {
        isPrime = false;
        break;
    }
```

Run faster? Why?
Prove?

```
if (isPrime == false)
...

```

Example 1(cnt.)

```
... // Declaration
if (n % 2 == 0) ...
for (i = 3; i <= sqrt(n); i += 2)
    if (n % i == 0) {
        isPrime = false;
        break;
    }
```

```
if (isPrime == false)
...

```

Run faster? Why?

Is it correct?

Fix it to run correctly?
(exercise)

Example 2

Given a number n , calculate the sum $S = 1 + 2 + \dots + n$

```
// Declaration
int i, n; cin >> n;

// Calculation
int res = 0;
for (i = 1; i <= n; i++)
    res += i;
```

```
// Declaration
int i, n; cin >> n;

// Calculation
int res = n * (n+1) / 2;
```

Which one runs faster?

Example 3

Given a number n , calculate the sum

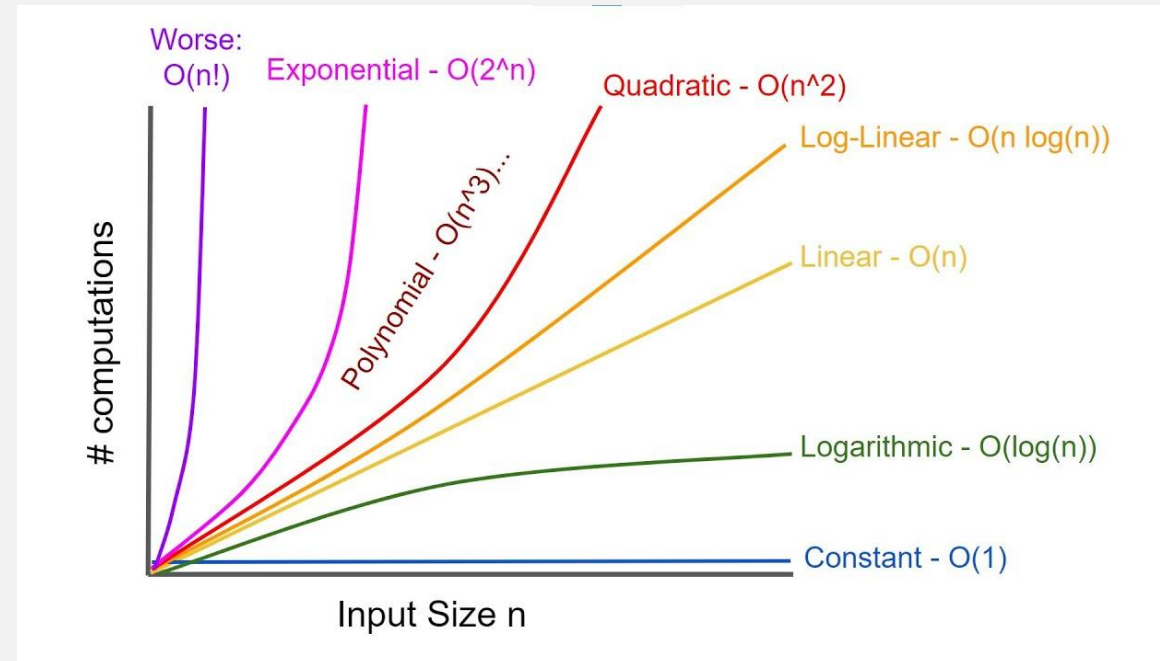
$$S = 1 + 1 + 1 + 2 + \dots + 1 + n + 2 + 1 + \dots + n + n$$

```
int res = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        res += i + j;
```

Any other approach
is better? (exercise)

Algorithm Efficiency

- Algorithm complexity (e.g., big-O)
 - Time complexity
 - Space complexity
- Major types of complexities
 - Constant: $O(1)$
 - Logarithmic: $O(\log(n))$
 - Linear: $O(n)$
 - Polynomial: $O(n^2)$, $O(n^3)$, ...
 - Exponential: $O(2^n)$
 - Factorial: $O(n!)$



Big O Complexity Chart

Details are in the course
"Data Structures and Algorithms"

Exercises

Exercises for branching structure

- 1) Input an integer `num`. Print out the name of a value if the value of the integer is 1 to 9, otherwise display "cannot pronounce".
- 2) Input a character `cha`, if it is lower case, convert it to upper case and vice versa.
- 3) Solve the equation $ax + b = 0$.
- 4) Solve the equation $ax^2 + bx + c = 0$.
- 5) Input three average scores of three students. Print out ranks of three students. If some scores are same, it is considered a same rank. For example: Input: 9 9 10. Output: 2 2 1

Exercises for branching structure

- 6) Input 4 integers `a`, `b`, `c` and `d`. Sorting them in an ascending order.
- 7) Calculate a taxi fee from a number `nKm`. Knowing some conditions:
 - a. The price of the first km: 15000đ
 - b. From 2 km to 5 km for each km: 13500đ
 - c. If it is over 6 km for each km: 11000đ
 - d. If the total is over 120 km, the bill will be reduced 10%
- 8) Input three numbers. Check if they are three sides of the triangle. If so, what type of triangle is it?
- 9) Input `month` and `year`. Return a number of days of this `month`

Exercises for loop structure

- 1) Input an unsigned integer n ($n > 0$). Check if:
 - a. It is a symmetrical number? E.g., 121, 12321, ...
 - b. It is a square number? E.g., 4, 9, 16, ...
 - c. It is a prime number? E.g., 2, 3, 5, 7, ...
 - d. What are the largest and smallest digits in n ? E.g., 124, max = 4, min = 1.
 - e. The digits in n are an ascending order or descending order or not

Exercises for loop structure

2) Input an unsigned integer n . Calculate:

a. $S = 1 + 2 + \dots + n$

b. $S = 1^2 + 2^2 + \dots + n^2$

c. $S = 1 + 1/2 + \dots + 1/n$

d. $S = 1 * 2 * \dots * n = n!$

e. $S = 1! + 2! + \dots + n!$

3) Input three unsigned integers a , b , and n , where $a, b < n$. Calculate sum of unsigned integers smaller than n and such integers are divided by a but not divided by b .

Exercises for loop structure

- 4) Calculate sum of prime numbers that are smaller than n
($0 < n < 50$)
- 5) Given an unsigned integer n . Print the digits in reverse. E.g.,
 $1706 \rightarrow 6071$
- 6) Find the greatest common divisor (GCD) of two unsigned integer numbers a and b . E.g., $a = 4$, $b = 6$, GCD of a and b is 2
- 7) Given n . Print n numbers in Fibonacci series